

halfedge：链式查询 DSL 设计文档

src/query.rs

halfedge 库

2026-07-04

目录

1	模块定位	2
2	MeshQuery<T> 核心结构	2
2.1	数据结构	2
2.2	三个核心方法	2
3	方法一览	3
3.1	HalfEdgeId 方法	3
3.2	VertexId 方法	3
3.3	MeshQuery<T> 链式方法	3
4	CW / CCW 旋转规则	4
4.1	几何图示	4
4.2	公式推导	4
5	链式求值流程	5
6	示例查询	5
6.1	用户示例：两三角形共享边上的 CW 旋转	5
6.2	闭合扇形上的连续旋转	6
6.3	面边界环遍历	6
6.4	src/dst 往返查询	6
7	健壮性	7
8	内部辅助函数	7
9	单元测试覆盖	7
10	设计取舍	8

1 模块定位

query.rs 在 storage.rs 之上叠加链式查询能力，灵感来自 SMesh 和 OpenMesh 的 Builder 模式查询接口。

核心思路：为 VertexId 和 HalfEdgeId 提供一组返回 MeshQuery<T> 的方法，每个方法返回一个新的 MeshQuery，捕获前序查询的闭包，在 .run(&mesh) 时一次性求值整条链。



2 MeshQuery<T> 核心结构

2.1 数据结构

```
1 #[must_use]
2 pub struct MeshQuery<T> {
3     f: Box<dyn FnOnce(&MeshStorage) -> Option<T>>,
4 }
```

MeshQuery<T> 内部存储一个 FnOnce 闭包，接收 &MeshStorage 并返回 Option<T>。闭包在构造时不执行，只在 .run() 时求值。

#[must_use] 属性 结构体标注 #[must_use] 后，任何返回 MeshQuery<T> 却未被消费（未调用 .run() 或链式方法）的值都会触发编译器警告。这可防止用户 忘记 .run() 导致静默丢弃查询：

```
1 //      unused MeshQuery that must be used
2 v0.halfedge_to(v1).cw_rotated();           //      .run(&mesh)
3
4 //      .run()
5 let nb = v0.halfedge_to(v1).cw_rotated().dst_vert().run(&mesh);
```

所有 HalfEdgeId / VertexId 直接方法与 MeshQuery<T> 链式方法均返回 MeshQuery<T>，因此均受此属性覆盖。

2.2 三个核心方法

方法	签名	语义
new(f)	F: FnOnce(&MeshStorage) -> Option<T>	从闭包构造查询
run(mesh)	self -> Option<T>	执行查询链，消费 self
then(f)	(self, F) -> MeshQuery<U>	内部组合子：链接下一步

then 是内部组合子，实现链式调用的核心：

```
1 fn then<F, U>(self, f: F) -> MeshQuery<U>
2 where
```

```

3  F: FnOnce(&MeshStorage, T) -> Option<U> + 'static,
4  U: 'static,
5  {
6      let prev = self.f;
7      MeshQuery::new(move |mesh| {
8          let val = prev(mesh)?;    //
9          f(mesh, val)              //
10     })
11 }

```

关键语义：前序闭包返回 `None` 时，`?` 运算符**短路**，后续步骤不执行，最终返回 `None`。

3 方法一览

3.1 HalfEdgeId 方法

方法	返回类型	拓扑操作
<code>twin()</code>	<code>MeshQuery<HalfEdgeId></code>	$h \mapsto h.twin$
<code>next()</code>	<code>MeshQuery<HalfEdgeId></code>	$h \mapsto h.next$
<code>prev()</code>	<code>MeshQuery<HalfEdgeId></code>	$h \mapsto h.prev$
<code>face()</code>	<code>MeshQuery<FaceId></code>	$h \mapsto h.face$
<code>src_vert()</code>	<code>MeshQuery<VertexId></code>	$h \mapsto h.twin.vertex$ (origin)
<code>dst_vert()</code>	<code>MeshQuery<VertexId></code>	$h \mapsto h.vertex$ (tip)
<code>cw_rotated()</code>	<code>MeshQuery<HalfEdgeId></code>	$h \mapsto h.twin.next$
<code>ccw_rotated()</code>	<code>MeshQuery<HalfEdgeId></code>	$h \mapsto h.prev.twin$

3.2 VertexId 方法

方法	返回类型	语义
<code>halfedge()</code>	<code>MeshQuery<HalfEdgeId></code>	取 v 的 outgoing 入口
<code>halfedge_to(w)</code>	<code>MeshQuery<HalfEdgeId></code>	遍历 outgoing 环找 $tip = w$ 的半边

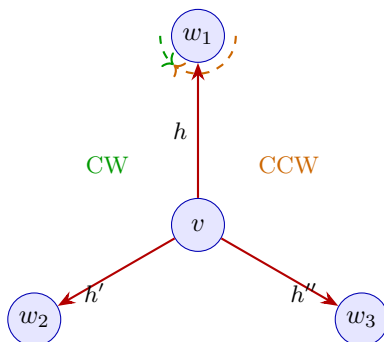
3.3 MeshQuery<T> 链式方法

`MeshQuery<HalfEdgeId>`、`MeshQuery<VertexId>` 与 `MeshQuery<FaceId>` 各自实现与对应 ID 类型相同的方法集，通过 `then` 组合子链接前序查询与当前步骤。

<code>MeshQuery<T></code>	可用方法
<code>MeshQuery<HalfEdgeId></code>	<code>twin / next / prev / face</code> <code>src_vert / dst_vert</code> <code>cw_rotated / ccw_rotated</code>
<code>MeshQuery<VertexId></code>	<code>halfedge / halfedge_to</code>
<code>MeshQuery<FaceId></code>	<code>halfedge</code> (取面入口半边，转入 <code>HalfEdgeId</code> 链)

4 CW / CCW 旋转规则

4.1 几何图示



4.2 公式推导

设 h 是顶点 v 的 outgoing 半边 ($h.twin.vertex = v$):

CW 旋转 (顺时针绕 v 到相邻面):

$$cw_rotated(h) = h.twin.next$$

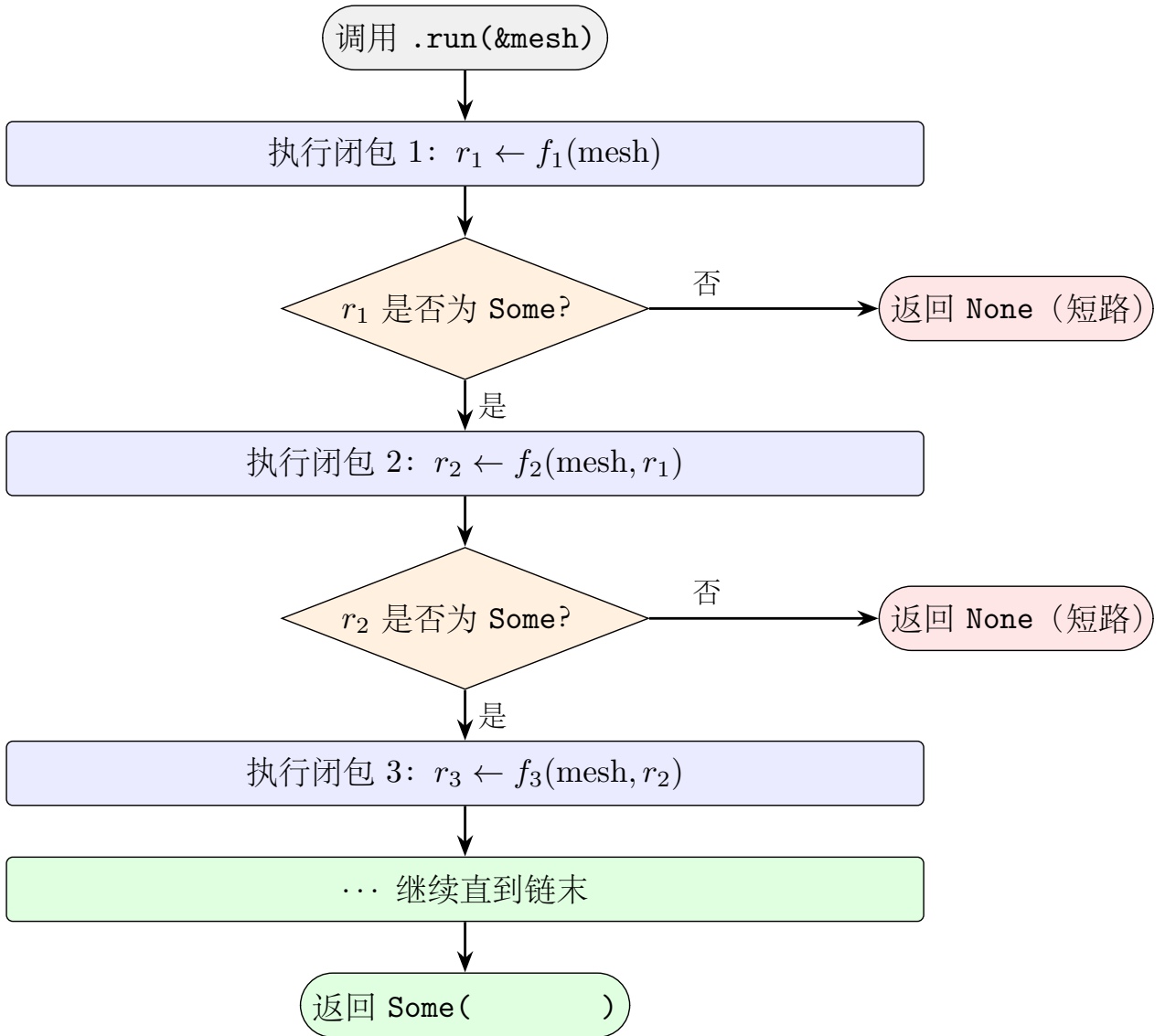
推导: $h.twin$ 结尾落在 v ; 其 $next$ 起始于 v , 属于 h 顺时针方向的相邻面。

CCW 旋转 (逆时针绕 v 到相邻面):

$$ccw_rotated(h) = h.prev.twin$$

推导: $h.prev$ 与 h 同面, 结尾落在 v ; 其 $twin$ 起始于 v , 属于 h 逆时针方向的相邻面。

5 链式求值流程

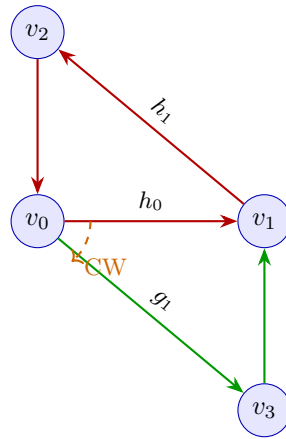


短路语义：链中任一环节返回 **None**，后续闭包不执行，最终返回 **None**。这保证了无效 ID、边界半边等场景下的安全行为。

6 示例查询

6.1 用户示例：两三角形共享边上的 CW 旋转

```
1 //                                v0-v1
2 // v0.halfedge_to(v1) = h0 (v0+v1,      )
3 // h0.cw_rotated() = twin(h0).next = g0.next = g1 (v0+v3)
4 // g1.dst_vert() = v3
5 let neighbor = v0.halfedge_to(v1)
6   .cw_rotated()
7   .dst_vert()
8   .run(&mesh);
9 assert_eq!(neighbor, Some(v3));
```



链: $h_0 \xrightarrow{cw} g_1 \xrightarrow{dst} v_3$

6.2 闭合扇形上的连续旋转

```

1 //      c  3
2 // a1: c+v0, cw_rotated(a1) = a3 (c+v2)
3 //      CW      3      a1
4 let result = a1.cw_rotated()
5     .cw_rotated()
6     .cw_rotated()
7     .run(&mesh);
8 assert_eq!(result, Some(a1));
9
10 // CCW
11 let result = a1.ccw_rotated()
12     .ccw_rotated()
13     .ccw_rotated()
14     .run(&mesh);
15 assert_eq!(result, Some(a1));

```

6.3 面边界环遍历

```

1 // h0.next().next().next()      h0
2 assert_eq!(h0.next().next().next().run(&mesh), Some(h0));
3
4 // h0.next().dst_vert() = h1.vertex = v2
5 assert_eq!(h0.next().dst_vert().run(&mesh), Some(v2));

```

6.4 src/dst 往返查询

```

1 // h0.src_vert() = v0, v0.halfedge_to(v1) = h0
2 //
3 assert_eq!(

```

```

4 h0.src_vert().halfedge_to(v1).run(&mesh),
5 Some(h0)
6 );

```

7 健壮性

- 无效 / 已删除 ID: 所有方法依赖 `mesh.get_halfedge` / `mesh.get_vertex`, 返回 `None` 时链短路, 不 panic。
- 边界半边: `twin.next` 或 `prev.twin` 撞到 `None` 时, `cw_rotated` / `ccw_rotated` 返回 `None`。
- 短路保护: 链中任一环节 `None`, 后续闭包不执行, 避免 `unwrap` / 空指针。
- `halfedge_to` 复用 `traversal::VertexRingLazy`, 正确处理闭合环与开链, 以半边总数加一为上界防止死循环。

8 内部辅助函数

每个半边操作都抽取为独立的辅助函数, 被 `HalfEdgeId` 直接方法和 `MeshQuery<HalfEdgeId>` 链式方法共用, 消除代码重复:

辅助函数	实现
<code>he_twin(mesh, he)</code>	<code>mesh.get_halfedge(he).and_then(h h.twin)</code>
<code>he_next(mesh, he)</code>	<code>mesh.get_halfedge(he).and_then(h h.next)</code>
<code>he_prev(mesh, he)</code>	<code>mesh.get_halfedge(he).and_then(h h.prev)</code>
<code>he_face(mesh, he)</code>	<code>mesh.get_halfedge(he).and_then(h h.face)</code>
<code>he_src_vert(mesh, he)</code>	<code>get(he).twin → get → .vertex</code>
<code>he_dst_vert(mesh, he)</code>	<code>get(he) → .vertex</code>
<code>he_cw_rotated(mesh, he)</code>	<code>get(he).twin → get → .next</code>
<code>he_ccw_rotated(mesh, he)</code>	<code>get(he).prev → get → .twin</code>
<code>vertex_halfedge(mesh, v)</code>	<code>get_vertex(v).and_then(vt vt.halfedge)</code>
<code>vertex_halfedge_to(mesh, v, w)</code>	<code>VertexRingLazy::new(mesh, v).find(...)</code>
<code>face_halfedge(mesh, f)</code>	<code>get_face(f).and_then(fc fc.halfedge)</code>

9 单元测试覆盖

`src/query.rs` 末尾共 19 个测试, 分五组:

测试名	覆盖点
基本查询: <i>VertexId</i>	
<code>vertex_halfedge_query</code>	<code>v.halfedge().run()</code> 取 outgoing 入口
<code>vertex_halfedge_to_finds_correct_edge</code>	<code>v.halfedge_to(w)</code> 找正确半边 + 不存在邻居
基本查询: <i>HalfEdgeId</i>	
<code>halfedge_twin_query</code>	twin 互指
<code>halfedge_next_prev_query</code>	面 CCW 环 next/prev
<code>halfedge_face_query</code>	面内 $\rightarrow$ Some(f), 边界 twin $\rightarrow$ None
<code>halfedge_src_dst_vert_query</code>	origin / tip 取顶点
<code>halfedge_rotation_on_closed_fan</code>	CW/CCW 旋转 + 连续 3 次闭合
<code>halfedge_cw_rotated_boundary_returns_none</code>	边界半边 CW 旋转 $\rightarrow$ None
基本查询: <i>FaceId</i>	
<code>face_halfedge_query</code>	<code>f.halfedge().run()</code> 取面入口半边
<code>face_query_on_invalid_face_returns_none</code>	无效 FaceId 返回 None
链式查询	
<code>chain_user_example_on_two_triangles</code>	用户示例: CW 旋转取邻居
<code>chain_closed_fan_rotations</code>	闭合扇 CW/CCW 旋转取邻居
<code>chain_traverses_face_boundary</code>	next 链遍历面环
<code>chain_src_dst_roundtrip</code>	src_vert $\rightarrow$ halfedge_to 往返
<code>chain_short_circuits_on_none</code>	链中 None 短路
<code>meshquery_face_chain_halfedge</code>	<code>MeshQuery<FaceId></code> 链式 <code>halfedge()</code>
<code>face_chained_query</code>	<code>FaceId</code> 链式查询完整路径
无效 / 已删除 ID	
<code>invalid_ids_return_none</code>	孤立 / 已删除 / 默认 ID 全方法返回 None
<code>chain_on_invalid_ids_short_circuits</code>	默认 ID 链式短路

`cargo test` 全模块共 371 个用例, 全部通过 (0 失败 / 0 警告)。

10 设计取舍

- 为何用 `Box<dyn FnOnce>` 而非泛型? 泛型参数会在链中嵌套展开 (每链一步增加一层 `Then<...>` 包装), 类型复杂度爆炸。`Box<dyn FnOnce>` 将类型擦除为单一 `MeshQuery<T>`, 链长度不影响类型签名。
- 为何 `FnOnce` 而非 `FnMut`? 每个方法消费 `self`, 将前序闭包 `move` 进新闭包, 是一次性操作。`FnOnce` 语义最精确。
- 为何用 `then` 组合子? 半边操作的逻辑 (如 `twin.next` 用于 CW 旋转) 在直接方法和链式方法中完全相同。`then` 将公共逻辑抽取为辅助函数, 两处共用, 消除重复。
- 为何 `MeshQuery<FaceId>` 仅提供 `halfedge`? `FaceId` 在半边结构中只有「取面入口半边」一种 $O(1)$ 操作。`f.halfedge().run(&mesh)` 返回 `Option<HalfEdgeId>`, 自然转入 `MeshQuery<HalfEdgeId>` 链继续查询。